Journal of Process Control 111 (2022) 35–45

Contents lists available at ScienceDirect

Journal of Process Control

journal homepage: www.elsevier.com/locate/jprocont

Chance constrained policy optimization for process control and optimization

Panagiotis Petsagkourakis [a], Ilya Orson Sandoval [b], Eric Bradford [c], Federico Galvanin [a], Dongda Zhang [d,b], Ehecatl Antonio del Rio-Chanona [b,*]

[a] Centre for Process Systems Engineering (CPSE), University College London, Torrington Place, London, UK
[b] Centre for Process Systems Engineering (CPSE), Imperial College London, UK
[c] Department of Engineering Cybernetics, Norwegian University of Science and Technology, Trondheim, Norway
[d] Centre for Process Integration, Department of Chemical Engineering and Analytical Science, The University of Manchester, UK

ARTICLE INFO

Article history:
Received 6 December 2020
Received in revised form 13 November 2021
Accepted 4 January 2022
Available online 1 February 2022

Keywords:
Policy search
Reinforcement Learning
Data-driven process control
Chance constraints
Bayesian Optimization

ABSTRACT

Chemical process optimization and control are affected by (1) plant-model mismatch, (2) process disturbances, and (3) constraints for safe operation. Reinforcement learning by policy optimization would be a natural way to solve this due to its ability to address stochasticity, plant-model mismatch, and directly account for the effect of future uncertainty and its feedback in a proper closed-loop manner; all without the need of an inner optimization loop. One of the main reasons why reinforcement learning has not been considered for industrial processes (or almost any engineering application) is that it lacks a framework to deal with safety critical constraints. Present algorithms for policy optimization use difficult-to-tune penalty parameters, fail to reliably satisfy state constraints or present guarantees only in expectation. We propose a chance constrained policy optimization (CCPO) algorithm which guarantees the satisfaction of joint chance constraints with a high probability — which is crucial for safety critical tasks. This is achieved by the introduction of constraint tightening (backoffs), which are computed simultaneously with the feedback policy. Backoffs are adjusted with Bayesian optimization using the empirical cumulative distribution function of the probabilistic constraints, and are therefore self-tuned. This results in a general methodology that can be imbued into present policy optimization algorithms to enable them to satisfy joint chance constraints with high probability. We present case studies that analyse the performance of the proposed approach.

1. Introduction

The optimization of chemical processes presents distinctive challenges to the stochastic systems community given that they suffer from three conditions: (1) there is no precise known model for most industrial scale processes (plant-model mismatch), leading to inaccurate predictions and convergence to suboptimal solutions, (2) the process is affected by disturbances (i.e. it is stochastic), and (3) state constraints must be satisfied due to operational and safety concerns, therefore constraint violation can be detrimental or even dangerous. In this work we use constrained policy search, a reinforcement learning (RL) technique, to address the above challenges.

RL is a machine learning technique that computes a policy which learns to perform a task by interacting with the (stochastic) environment. RL has been shown to be a powerful control approach, and one of the few control techniques able to handle nonlinear stochastic optimal control problems [1,2]. RL in the approximate dynamic programming (ADP) sense has been studied for chemical process control in [3]. A model-based strategy and a model-free strategy for control of nonlinear processes were proposed in [4]. ADP strategies were used to address fed-batch reactor optimization, in [5] mixed-integer decision problems were addressed with applications to scheduling. In [6] RL was combined with distributed optimization techniques, to solve an input-constrained optimal control, among other works (e.g. [7,8]). All these approaches rely on action-value methods, which approximate the solution of the Hamilton–Jacobi–Bellman equation (HJBE), and have been shown to be reliable for some problem instances. RL also shares similar features with multiparametric model predictive control (or explicit MPC) [9,10], where an explicit representation of the controller can be found, such that it satisfies the optimality conditions. However, explicit MPC usually requires some approximations or assumptions on the models.

* Corresponding author.
E-mail address: a.del-rio-chanona@imperial.ac.uk (E.A.d. Rio-Chanona).

https://doi.org/10.1016/j.jprocont.2022.01.003
0959-1524/

Policy gradient RL methods [11] have been proposed to directly optimize the control policy. Unlike action-value RL methods where convergence to local optima is not guaranteed, policy gradient methods can guarantee convergence to local optimality even in high dimensional continuous state and action spaces. However, the inclusion of constraints in policy gradient methods is not straightforward. Existing methods cannot guarantee strict feasibility of the policies even when initialized with feasible initial policies [12]. The main approaches to incorporate constraints make use of trust-region, fixed penalties [13,14], and cross entropy [12]. Furthermore, if online optimization is to be avoided, addressing the constraints by the use of penalties is a natural choice. Various approaches have been proposed in this direction, however, current approaches easily lose optimality or feasibility [13] and guarantee feasibility only in expectation. Following this thread of thought, a Lyapunov-based approach is implemented in [15], where a Lyapunov function is constructed and the unconstrained policy is projected to a safety layer allowing the satisfaction of constraints in expectation. In [16] an upper bound on the expected constraint violation is provided for the projected-based policy optimization, where the initial unconstrained policy is projected back to the constraint set. In [17] an interior-point inspired method that is widely used in control [18] is proposed, where constraints are incorporated into the reward using a logarithm barrier function, allowing the satisfaction of the constraints in expectation.

The above methods all guarantee constraint satisfaction in expectation, which is inadequate for safety critical engineering applications (very loosely speaking, this means violating 50% of the time). Furthermore, although penalties are a natural way to avoid an online optimization loop (one of the main advantages of policy gradients), tuning these penalties is not always straightforward, and usually rely on heuristics which significantly affect the performance of policy gradient algorithms [19]. This suggests constrained policy gradient methods would benefit from self-tuning parameters. As mentioned earlier, it is well known that policy gradient methods present many advantages, however, their application domain will remain limited until they can handle constraint satisfaction reliably. This is the main challenge addressed in this work. Our proposed method – chance constrained policy optimization (CCPO) – trains a policy to guarantee the satisfaction of joint chance constraints. The proposed method allows for the satisfaction of constraints with a high probability rather than only in expectation. To achieve the satisfaction of joint chance constraints without the need of online optimization, we tighten constraints. To tighten the constraints we transform the problem of finding the tightening that satisfies the probabilistic constraints as a root-finding problem, which we address by reformulating it as an expensive black box optimization problem. Therefore, the resulting problem can be solved efficiently by existing black-box optimization methods, in our case Bayesian optimization. Upon convergence, the proposed algorithm finds an optimal policy that guarantees the constraint satisfaction to the desired tolerance.

The structure of this paper is as follows. The problem statement is outlined in Section 2, the details of the proposed method for probabilistic satisfaction in RL is presented in Section 3. A case study is presented in Section 4, where the framework is applied to a dynamic bioreactor system, and in the last section, conclusions are outlined.

2. Problem statement

In this work, the dynamic system is assumed to be given by a probability distribution, following a Markov process,

$$\mathbf{x}_{t+1} \sim p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t), \quad \mathbf{x}_0 \sim p(\mathbf{x}_0), \tag{1}$$

where $p(\mathbf{x}_i)$ is the probability density function of $\mathbf{x}_i$, with $\mathbf{x} \in \mathbb{R}^{n_x}$ representing the states, $\mathbf{u} \in \mathbb{R}^{n_u}$ the control inputs and t discrete time. This behaviour is observed in systems when stochastic disturbances are present and/or other uncertainties affect the physical system, like parametric uncertainties. A discrete-time system with disturbances and parametric uncertainties can be written as:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t, \mathbf{p}, \mathbf{w}_t) \tag{2}$$

where $\mathbf{w} \in \mathbb{R}^{n_w}$ is a vector of disturbances and $\mathbf{p} \in \mathbb{R}^{n_p}$ are uncertain parameters. This can be represented by (1). In this work we seek to maximize an objective function in expectation by using an optimal stochastic policy subject to probabilistic constraints despite the uncertainty of the system. This problem can be written as a stochastic optimal control problem (SOCP):

$$\mathcal{P}(\pi(\cdot)) := \begin{cases} \max\limits_{\pi(\cdot)} \mathbb{E}\left(J(\mathbf{x}_0, \ldots, \mathbf{x}_T, \mathbf{u}_0, \ldots, \mathbf{u}_T)\right) \\ \text{s.t.} \\ \mathbf{x}_0 \sim p(\mathbf{x}_0) \\ \mathbf{x}_{t+1} \sim p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) \\ \mathbf{u}_t \sim p(\mathbf{u}_t|\mathbf{x}_t) = \pi(\mathbf{x}_t) \\ \mathbf{u}_t \in \mathbb{U} \\ \mathbb{P}(\bigcap\limits_{i=0}^{T}\{\mathbf{x}_i \in \mathbb{X}_i\}) \geq 1-\alpha \\ \forall t \in \{0, \ldots, T-1\} \end{cases} \tag{3}$$

where J is the objective function, $\mathbb{U}$ denotes the set of hard constraints for the control inputs, $\mathbb{X}_i$ represents constraints for states that must be satisfied with a probability $1-\alpha$. Specifically,

$$\mathbb{X}_t = \{\mathbf{x}_t \in \mathbb{R}^{n_x} | g_{j,t}(\mathbf{x}_t) \leq 0, j = 1, \ldots, n_g\}, \tag{4}$$

with $g_{j,t}$ being the jth constraint to be satisfied at time instant t and the joint chance (also called probabilistic) constraints ($\mathbb{P}(\bigcap_{t=0}^{T}\{\mathbf{x}_t \in \mathbb{X}_i\}) \geq 1-\alpha$) are satisfied for the full trajectory over all $t \in \{0, \ldots, T\}$. The probability density function of $\mathbf{u}_t$ given state $\mathbf{x}_t$ is $p(\mathbf{u}_t|\mathbf{x}_t)$, and $\pi(\mathbf{x}_t)$ is the state feedback policy. Additionally, $\pi(\cdot)$ is the stochastic policy. Unfortunately, this SOCP is generally intractable and approximations must be sought, hence the use of RL [1,20,21].

In RL a policy $\pi_\theta(\cdot)$ parametrized by the parameters θ is constructed. This policy maximizes the expectation of the objective function $J(\cdot)$. In the finite horizon discrete-time case, this objective function can be defined as

$$J = \sum_{t=0}^{T} \gamma^t R_t(\mathbf{u}_t, \mathbf{x}_t), \tag{5}$$

where $\gamma \in [0, 1]$ is the discount factor and R_t a given reward at the time instance t for the values of $\mathbf{u}_t$, $\mathbf{x}_t$.

Notice that we seek for the policy to satisfy joint chance constraints with some high probability. Previous approaches have address the satisfaction of constraints in expectation; this means that constraints are violated (roughly speaking) half of the time.

3. Constrained policy optimization for chance constraints

In RL agents take actions to maximize some expected reward given a performance metric. In process control, these agents become the controller, which use a feedback policy $\pi(\cdot)$, to optimize the expected value of an economic metric of the process (J). The physical system (or the model) at each sampling time produces a value for the reward R which reflects the performance of the policy. The RL algorithm determines the feedback policy that

produces the greatest reward in expectation, which is referred to as policy optimization. However, there is no natural way to handle constraints, and a constraint satisfied in expectation may still have a very high probability of not being satisfied. To satisfy the constraints with some high probability and not only in expectation, we tighten the constraint with backoffs [22,23] $b_{j,t}$ as:

$$\overline{\mathbb{X}}_t = \{\mathbf{x}_t \in \mathbb{R}^{n_x} | g_{j,t}(\mathbf{x}_t) + b_{j,t} \leq 0, j = 1, \ldots, n_g\}, \tag{6}$$

where variables $b_{j,t} \geq 0$ represent the backoffs which tighten the original feasible set $\mathbb{X}_t$ defined in (4). Backoffs restrict the perceived feasible space by the controller, and allow guarantees on the satisfaction of chance constraints.

We denote $\boldsymbol{\tau}$, as the joint random variable of states, controls and rewards for a trajectory with a time horizon T:

$$\boldsymbol{\tau} = (\mathbf{x}_0, \mathbf{u}_0, R_0, \ldots, \mathbf{x}_{T-1}, \mathbf{u}_{T-1}, R_{T-1}, \mathbf{x}_T, R_T). \tag{7}$$

We also assume that the policy can be parametrized by a finite number of parameters θ, and denote this parametrized policy as:

$$\mathbf{u}_t \sim p(\mathbf{u}_t|\mathbf{x}_t) = \pi_\theta(\mathbf{x}_t, D_t), \quad \mathbf{u}_t \in \mathbb{U}, \tag{8}$$

where $\mathbf{u}_t \in \mathbb{U}$ is inherently satisfied by the construction of the policy, e.g. the policy passes through a bounded and differential squashing function [24], and D_t is a window of past inputs and states that are used by the policy, for example, if the parametrized policy is a recurrent neural network, then D_t corresponds to a number of past states and controls.

Recently a methodology was proposed that satisfies the expected value of the constraints [12,14], however this is not adequate for safety critical constraints in chemical processes. Instead, to account for constraint violations, we incorporate probabilistic constraints. The problem can then be reformulated as:

$$\begin{aligned}
\pi_{\theta^*} = &\arg\max_{\pi_\theta(\cdot)} \mathbb{E}_{\boldsymbol{\tau}}\left(J(\boldsymbol{\tau})\right) \\
&s.t. \\
&\mathbf{u}_t \sim p(\mathbf{u}_t|\mathbf{x}_t) = \pi_\theta(\mathbf{x}_t, D_t), \ \forall t \in 0, \ldots, T-1 \\
&\mathbf{u}_t \in \mathbb{U} \\
&\boldsymbol{\tau} \sim p(\boldsymbol{\tau}|\theta) \\
&\mathbb{P}_{\boldsymbol{\tau}}\left(\bigcap_{i=1}^{T}\{\mathbf{x}_i \in \mathbb{X}_i\}\right) \geq 1-\alpha
\end{aligned} \tag{9}$$

where $p(\boldsymbol{\tau}|\theta)$ represents the probability of the trajectory $\boldsymbol{\tau}$ given the parametrization θ of the feedback policy, and π_{θ^*} is the optimal policy.

In order to solve (9) we propose to:

(1) Parameterize the feedback policy by a multilayer neural network that computes the mean and variance of the control actions, which are consequently sampled as a normal distribution resulting in a stochastic policy.
(2) The probabilistic constraint in (9) is substituted by a tightened constraint set $\overline{\mathbb{X}}_t$ to guarantee closed-loop probabilistic constraint satisfaction.
(3) The tightened constraints $g_{j,t}(\mathbf{x}_t) + b_{j,t} \leq 0, j = 1, \ldots, n_g$ are incorporated into the objective function with a penalty function or an augmented Lagrangian function. This avoids the need to explicitly solve an optimization problem at run-time.
(4) The policy optimization is performed by a policy gradient framework [11].
(5) The value of the backoffs should be the smallest value that guarantees constraint satisfaction with a probability of at least $1-\alpha$. This is formulated as a black-box optimization problem, which can be solved efficiently by existing methods [25–27].

Notice that the value of the backoffs imply a trade-off: large values guarantee constraint satisfaction, but they make the problem over-conservative and mitigate performance, while smaller values produce solutions with high rewards, but may not guarantee the constraint satisfaction to the desired accuracy $(1-\alpha)$. Also notice that if backoffs are too large, the problem might become infeasible.

In the next subsections we introduce the components that were outlined above.

3.1. Policy parametrization

For the policy parametrization we use recurrent neural networks, (RNNs) [28,29]. Let N be the length of the window of past states and controls to be used by the parametrized feedback policy, i.e. $D_t = \left[\mathbf{x}_{t-1}^T, \mathbf{u}_{t-1}^T, \mathbf{x}_{t-2}^T, \mathbf{u}_{t-2}^T, \ldots, \mathbf{u}_{t-N-1}^T\right]^T$. Then the stochastic policy can be defined as:

$$\pi_\theta(\mathbf{x}_t, D_t) = \mathcal{N}(\mathbf{u}_t|\boldsymbol{\mu}_t^{\mathbf{u}}, \boldsymbol{\Sigma}_t^{\mathbf{u}}), \quad [\boldsymbol{\mu}_t^{\mathbf{u}}, \boldsymbol{\Sigma}_t^{\mathbf{u}}] = s_\theta(\mathbf{x}_t, D_t) \tag{10}$$

where s_θ is the multilayer RNN parametrized by θ, and $\boldsymbol{\mu}_t^{\mathbf{u}}$ and $\boldsymbol{\Sigma}_t^{\mathbf{u}}$ are the mean and covariance of the normal distribution from which the control $\mathbf{u}_t$ is drawn. Deep structures are employed to enhance the performance of the learning process [30,31].

3.2. Probabilistic constraints

In this section, we present a strategy to handle (joint) chance constraints by policy gradient methods. There is no closed form solution for general probabilistic constraints, and therefore we compute their empirical cumulative distribution function (ECDF) instead. To guarantee with high probability (and not only in expectation) the satisfaction of constraints, we introduce constraint tightening by using backoffs $b_{j,t}$ (see Eq. (6)). Therefore, we conduct closed-loop Monte Carlo (MC) simulations to compute the backoffs. We pose the problem of attaining the smallest backoffs (least restrictive) that still satisfy our constraints with a probability of at least $1-\alpha$ as an expensive black-box optimization problem, which is solved by Bayesian optimization. In this way we compute a policy that satisfies our constraints with a probability of at least $1-\alpha$, and we can be certain of this with a confidence of at least $1-\epsilon$. The rest of this subsection details the aforementioned methodology.

For convenience we define a single-variate random variable $C(\cdot)$ representing the satisfaction of the joint chance constraint:

$$C(\mathbf{X}) = \max_{(j,t)\in\{1,\ldots,n_g\}\times\{1,\ldots,T\}} g_{j,t}(\mathbf{x}_t) \tag{11}$$

$$F(c) = \mathbb{P}\left(C(\mathbf{X}) \leq c\right) \tag{12}$$

then

$$F(c := 0) = \mathbb{P}\left(C(\mathbf{X}) \leq 0\right) = \mathbb{P}\left(\bigcap_{t=0}^{T}\{\mathbf{x}_t \in \mathbb{X}_t\}\right), \tag{13}$$

where $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_T]^{\mathsf{T}}$, and F is the cumulative distribution function (CDF). There is no analytical expression for general probabilistic constraints, and we therefore approximate them by a non-parametric approximation, i.e. their empirical cumulative distribution function (ECDF). We can approximate the value of $F(0)$ by its sample approximation on S Monte Carlo (MC) simulations of $\mathbf{X}$:

$$F(0) \approx F_S(0) = \frac{1}{S}\sum_{s=1}^{S} \mathbb{1}\left(C(\mathbf{X}^s) \leq 0\right), \tag{14}$$

where $\mathbf{X}^s = [\mathbf{x}_1^s, \ldots, \mathbf{x}_T^s]^{\mathsf{T}}$ is the sth MC sample of the state trajectory. Notice that $F_S(0)$ is a random variable. Define $\mathbb{1}\{C(\mathbf{X}) \leq$

0} as the indicator function for a single trajectory to satisfy all constraints:

$$\mathbb{1}\{C(\mathbf{X}^s) \leq 0\} = \begin{cases} 1, & C(\mathbf{X}^s) \leq 0 \\ 0, & otherwise. \end{cases}$$

Notice that $F_S(0)$ follows a Binomial distribution, as the indicator function is a Bernoulli random variable. Hence, $F_S(0) \sim \frac{1}{S}\text{Bin}(S, F(0))$. Later on we will use a realization from this random variable (i.e. $\hat{F}_S$) to approximate the probability for a trajectory to satisfy all constraints. The confidence bound for the ECDF can then be computed from the Binomial CDF via the Clopper–Pearson interval [22,32,33].

Notice that the quality of the approximation in (14) strongly depends on the number of samples S used and it is therefore desirable to quantify the uncertainty of the sample approximation itself. This problem has been studied to a great extent in the statistics literature [32,33], leading to the following lemma.

Lemma 1 ([33]). *Given the random variable $F_S(0)$ (the ECDF — see (14)) based on S i.i.d. samples, the true value of the CDF, $F(0)$, has the following lower bound F_{lb} with a confidence level of $1-\epsilon$:*

$$\begin{aligned} &\mathbb{P}(F(0) \geq F_{lb}) \geq 1-\epsilon, \\ &F_{lb} = 1 - \text{betainv}(\epsilon, S+1-S\,F_S(0), S\,F_S(0)), \end{aligned} \tag{15}$$

with betainv$(\cdot,\cdot,\cdot)$ *being the inverse of the beta CDF with parameters $\{S+1-S\,F_S(0)\}$ and $\{S\,F_S(0)\}$.*

Lemma 1 states that it is possible to compute a probabilistic lower bound, with an arbitrary confidence of at least $1-\epsilon$, for the ECDF of the chance constraints. The following lemma follows directly using the above result.

Lemma 2 ([22]). *Given a realization $\hat{F}_S$ of the ECDF random variable $F_S(0)$ based on S independent samples, we can compute a corresponding realization of the random variable F_{lb} which is denoted $\hat{F}_{lb}$. If the value of $\hat{F}_{lb} \geq 1-\alpha$, then, with a confidence level of $1-\epsilon$, our original chance constraint in Eq. (13) holds.*

The implication of Lemma 2 is that we can guarantee the satisfaction of joint chance constraints with a user-defined probability of at least $1-\alpha$ and a user-defined confidence of at least $1-\epsilon$. Values of $\hat{F}_{lb}$ that are higher than $1-\alpha$ lead to more conservative solutions, and consequently worse values for the objective function. On the other hand, when $\hat{F}_{lb}$ is lower than $1-\alpha$, then the objective attains better values, but the constraints are not satisfied to the required degree. Therefore, the best trade-off is realized if $\hat{F}_{lb}$ is as close as possible to $1-\alpha$.

Hence, we aim to compute tightened constraints employing backoffs $b_{j,t}$ such that $\hat{F}_{lb} - (1-\alpha)$ is close to 0 when the policy optimization has terminated. Lemma 2 allows us to find a solution to the original problem having satisfied all joint chance constraints with a confidence at least $1-\epsilon$.

To compute the tightened constraint set as denoted in Eq. (6), we first compute an initial set of backoffs ($b^0_{j,t}$), as it has been proposed in [34], where

$$\mathbb{E}\left(g_{j,t}(\mathbf{x}_t)\right) + b^0_{j,t} = 0 \;\text{ gives }\; \mathbb{P}\left(g_{j,t}(\mathbf{x}_t) \leq 0\right) \geq 1-\delta \quad \forall j,t, \tag{16}$$

with δ being a tuning parameter for the initial backoffs $b^0_{j,t}$. Additionally, the expected value of $g_{j,t}$ is approximated with a sample average approximation (SAA):

$$\bar{g}_{j,t} = \frac{1}{S}\sum_{s=1}^{S} g_{j,t}(\mathbf{x}^s_t). \tag{17}$$

The initial backoff values are computed to probabilistically satisfy each constraint:

$$\mathbb{P}\left(g_{j,t}(\mathbf{x}_t) \leq 0\right) \geq 1-\delta, \tag{18a}$$

$$b^0_{j,t} = F^{-1}_{\mathbf{g}_{j,t}}(1-\delta) - \bar{g}_{j,t}(\mathbf{x}_t), \;\; \forall j,t \tag{18b}$$

with $\mathbf{g}_{j,t} = [g_{j,t}(\mathbf{x}^1_t), \ldots, g_{j,t}(\mathbf{x}^S_t)]^\intercal$ and $F^{-1}_{\mathbf{g}_{j,t}}(1-\delta)$ is the $1-\delta$ quantile of $\mathbf{g}_{j,t}$ (18a).

We wish the backoffs $\mathbf{b}$ (with elements $b_{j,t}\ \forall\, j,t \in \{1,\ldots,n_g\} \times \{1,\ldots,T\}$) to be large enough to ensure the probabilistic satisfaction of constraints. However, if the backoffs are too large it may result in a conservative solution, therefore a worse performance, or even infeasibility of the problem. To obtain the least conservative solution that still guarantees the probabilistic constraint satisfaction specified by $1-\alpha$ we solve a root-finding problem using Lemma 1 to find a vector $\boldsymbol{\gamma}$ that parametrizes $b_{j,t} := \gamma_j\, b^0_{j,t}\ \forall j,t$ such that

$$\hat{F}_{lb}(\boldsymbol{\gamma}) - (1-\alpha) = 0 \tag{19}$$

Notice, that now $\hat{F}_{lb}$ is a function of $\boldsymbol{\gamma} = [\gamma_1, \ldots, \gamma_{n_g}]$. In fact the Eq. (19) is an ideal scenario, where the lower bound could be forced to be $1-\alpha$. Here we try to minimize the distance between $\hat{F}_{lb}$ and $1-\alpha$. With the above procedure we compute a deterministic surrogate for the constraints that allows us to satisfy the joint chance constraints in the original optimization problem (9). In the subsequent section we explain how this constraint surrogate is incorporated into the reinforcement learning framework.

4. CCPO: Chance constrained policy optimization

4.1. Policy gradient for fixed backoffs

In this work we reformulate chance constraints such that their satisfaction is guaranteed with high probability, and they can be incorporated into the objective of the policy gradient method [11]. We therefore avoid the need for a numerical optimization every time the agent/controller outputs an action/control input.

Loosely speaking policy gradient methods aim to update the parametrized policy using the gradient of the reward. Without loss of generality, we outline the implementation for the REINFORCE [35] algorithm for ease of presentation, but this can be generalized to any policy gradient or actor–critic method [20,36].

Previous works have embedded Lagrangian methods as adaptive penalty coefficients to enforce satisfaction of constraints [14], however, an adaptive scheme such as gradient ascent or its variants on inequality multipliers are difficult to justify in theory (see for example [37] chapter 12 or [38] chapter 5), and in practice tend to have numerical issue when an arbitrary number of constraints are enforced and different constraints are active in different instances [37].

In this work, we instead propose a p-norm :

$$\hat{J}(\boldsymbol{\tau}, \mathbf{b}) = J(\boldsymbol{\tau}) - \kappa \sum_{t=1}^{T} \| [\mathbf{g}_t(\mathbf{x}_t) + \mathbf{b}_t]^- \|_p^p, \tag{20}$$

where $\left[g_{j,t}(\mathbf{x}_t) + b_{j,t}\right]^- = \max\{g_{j,t}, (\mathbf{x}_t) + b_{j,t}\ , 0\}$, $\|\cdot\|_p$ is a p-norm of the vector $\mathbf{g}_t = \left[g_{1,t}, \ldots, g_{n_g,t}\right]$. We advocate for the use of l_1 ($p=1$) and l_2 ($p=2$) norms due to the arbitrary number of constraints that may be considered and their numerical stability [37].

The following theorem can be stated for the satisfaction of the constraints given S Monte Carlo trajectories.

Theorem 3. *Consider the chance constrained stochastic optimal problem (9) and let π_{θ^*} be the trained policy that maximizes the expected reward (see [13,14] or (20)) and satisfies (19) using backoffs* **b**. *Then the joint chance constraints (3) will be satisfied with a confidence level of* $1-\epsilon$.

Proof. Assume that a policy π_{θ^*} is evaluated on the chance constraints (4) and a realization of the ECDF $\hat{F}_S$ is computed which leads to a realization $\hat{F}_{lb}$ of the random variable F_{lb}. Then, given Lemma 2, if $\hat{F}_{lb} \geq 1-\alpha$ policy π_{θ^*} will satisfy (13) with confidence $1-\epsilon$. □

Remark 1. Further a posteriori analysis can be performed in the system following the work of [39]. Given the number of trajectories S and a confidence level $1-\epsilon_S$, then the probability of constraint violation can be found to be bounded with a confidence $1-\epsilon_S$. More details can be in [39]

Remark 2. The feasibility of the constraints will guaranteed independently from the selection of κ (see Theorem 3).

Remark 3. The parameter κ can be updated using standard techniques from constrained optimization [37].

We now use the policy gradient theorem [11] to obtain an explicit gradient estimate of the reward with respect to the parameters of our policy:

$$\nabla_\theta \mathbb{E}_{\boldsymbol{\tau}}\left(\hat{J}\right) \approx \frac{1}{S}\sum_{s=1}^{S}\left[\hat{J}(\boldsymbol{\tau}^s,\mathbf{b})\nabla_\theta \sum_{t=0}^{T-1}\log\left(\pi_\theta(\mathbf{x}_t^s, D_t^s)\right)\right] \tag{21}$$

where $\boldsymbol{\tau}^s = (\mathbf{x}_0^s, \mathbf{u}_0^s, R_0^s, \ldots, \mathbf{x}_{T-1}^s, \mathbf{u}_{T-1}^s, R_{T-1}^s, \mathbf{x}_T^s, R_T^s)$ denotes the realization of the sth trajectory with R^s being the reward for sample s and

$$D_t^s = \left[(\mathbf{x}_{t-1}^s)^T, (\mathbf{u}_{t-1}^s)^T, \ldots, (\mathbf{u}_{t-N-1}^s)^T\right]^T \tag{22}$$

denotes the past states and controls used by the policy for sample s. The variance of the gradient estimate can be reduced with the aid of an action-independent baseline $\bar{\beta}_S$, which does not introduce a bias [40]. A simple but effective baseline is the expectation of reward under the current policy, approximated by the mean of the sampled paths:

$$\bar{\beta}_S = \frac{1}{S}\sum_{s=1}^{S}\hat{J}(\boldsymbol{\tau}^s,\mathbf{b}), \tag{23}$$

which leads to:

$$\nabla_\theta \mathbb{E}_{\boldsymbol{\tau}}\left(\hat{J}\right) \approx \frac{1}{S}\sum_{s=1}^{S}\left[\left(\hat{J}(\boldsymbol{\tau}^s,\mathbf{b}) - \bar{\beta}_S\right)\nabla_\theta \sum_{t=0}^{T-1}\log\left(\pi_\theta(\mathbf{x}_t^s, D_t^s)\right)\right]. \tag{24}$$

Using the above gradient of the expected reward with respect to the parameters the policy can now be iterative adjusted, in a steepest ascent framework or any of its variants (e.g. Adam):

$$\theta_{k+1} := \theta_k + \frac{\ell_k}{S}\sum_{s=1}^{S}\left[\left(\hat{J}(\boldsymbol{\tau}^s,\mathbf{b}) - \bar{\beta}_S\right)\nabla_\theta \sum_{t=0}^{T-1}\log\left(\pi_\theta(\mathbf{x}_t^s, D_t^s)\right)\right], \tag{25}$$

where ℓ_k is the adaptive learning rate. The algorithm that trains the policy network for a fixed backoff value **b** is outlined in Algorithm 1.

Algorithm 1 Policy gradient for fixed backoff

Input: Given optimization problem in Eq. (9) and modified objective in Eq. (20), initialize policy π_θ with parameters $\theta := \theta_0$, initial learning rate ℓ_0, learning rate update rule $L(\ell_k)$, number of samples for the gradient approximation S, number of epochs K, tolerance *tol*, and backoffs **b**.

for $k = 1$ **to** K **do**

1. Collect $\boldsymbol{\tau}^s$ and $\hat{J}(\boldsymbol{\tau}^s, \mathbf{b})$ for samples $s = 1, \ldots, S$ using policy θ_k.
2. Update policy π_θ: $\theta_{k+1} := \theta_k +$ $+\frac{\ell_k}{S}\sum_{s=1}^{S}\left[\left(\hat{J}(\boldsymbol{\tau}^s,\mathbf{b}) - \bar{\beta}_s\right)\nabla_\theta \sum_{t=0}^{T-1}\log\left(\pi_\theta(\mathbf{x}_t^s, D_t^s)\right)\right]$.
3. Update learning rate $\ell_{k+1} := L(\ell_k)$.
4. if $|\bar{\hat{J}}_{k+1} - \bar{\hat{J}}_k| \leq tol$ then *exit*, where $\bar{\hat{J}}$ refers to the sample average of $\hat{J}(\boldsymbol{\tau}^s, \mathbf{b})$.

end for

Output: Optimal policy π_{θ^*}, with $\theta^* := \theta_{k+1}$.

4.2. Backoff iterations

Solving Eq. (19) is complicated because there is no closed form solution, or even an explicit expression. To solve the root-finding problem in Eq. (19) efficiently, we formulate it as a least-squares expensive black box optimization problem [26] and solve it via Bayesian optimization with the objective function

$$\mathcal{F}(\boldsymbol{\gamma}) := \left(\hat{F}_{lb}(\boldsymbol{\gamma}) - (1-\alpha)\right)^2. \tag{26}$$

Algorithm 1 yields an optimal stochastic policy for fixed values of the backoffs. We propose Algorithm 2 to iteratively adjust the backoffs to guarantee probabilistic constraint satisfaction.

A description of the steps conducted in Algorithm 2 is presented here:

Step (1): The policy is trained with $\mathbf{b} = 0$ using Algorithm 1.

Step (2): The initial estimates of the backoff are computed by Monte-Carlo closed-loop simulations.

Step (3):

(3i) An initial set of backoffs parameter values $\boldsymbol{\Gamma} = [\boldsymbol{\gamma}^1, \ldots, \boldsymbol{\gamma}^{N_\Gamma}]$ are used to construct a set of optimal policies $\pi_{\theta^*}(\cdot|\boldsymbol{\gamma}^1)\ldots\pi_{\theta^*}(\cdot|\boldsymbol{\gamma}^{N_\Gamma})$, recall that $\boldsymbol{\gamma}$ parametrizes $b_{j,t} := \gamma_j\, b_{j,t}^0\ \forall j, t$.

(3ii) For each optimal policy $\pi_{\theta^*}(\cdot|\boldsymbol{\gamma}^i), i \in \{1, \ldots, N_\Gamma\}$, the lower bound on the ECDF, $\hat{F}_{lb}(\boldsymbol{\gamma}^i), i \in \{1, \ldots, N_\Gamma\}$, is computed by Monte Carlo (a cross-validation of sorts), along with the squared residual in Eq. (26). This yields a set of backoff parameter values $\boldsymbol{\Gamma} = [\boldsymbol{\gamma}^1, \ldots, \boldsymbol{\gamma}^{N_\Gamma}]$ that correspond to residuals $\mathbf{F} = [\mathcal{F}(\boldsymbol{\gamma}^1), \ldots, \mathcal{F}(\boldsymbol{\gamma}^{N_\Gamma})]$ for Eq. (26). This is the initial sample set used to map backoff values to residual values via a Gaussian process regression, and subsequently solved via a Bayesian optimization framework to enforce Eq. (19).

Step (4): For each mth iteration:

(4i) A Gaussian process which maps the backoff values to the squared residuals in Eq. (26) is constructed. We compute the objective function value as in Eq. (26) to find $\mathcal{F}(\boldsymbol{\gamma}^{N_\Gamma+m-1}) := \left(\hat{F}_{lb}(\boldsymbol{\gamma}^{N_\Gamma+m-1}) - (1-\alpha)\right)^2$.

(4ii) We conduct a Bayesian optimization step, via lower confidence bound minimization [41] of the GP. This allows us to obtain the next value for $\boldsymbol{\gamma}^{N_\Gamma+m}$.

(4iii) We compute a new policy $\pi(\cdot|\boldsymbol{\gamma}^{N_\Gamma+m})$ using Algorithm 1.

(4iv) We compute the new backoff **b** with the new value of the backoff parameter $\boldsymbol{\gamma}^{N_\Gamma+m}$. We also compute the new value for the residuals (objective) $\mathcal{F}$. Data matrices $\boldsymbol{\Gamma}$ and **F** are updated. The algorithm goes back to **(i)** where a new GP is constructed incorporating the new values of $\boldsymbol{\gamma}^{N_\Gamma+m}$ and $\mathcal{F}$ and the algorithm proceeds until some tolerance is achieved (e.g. $\mathcal{F}(\boldsymbol{\gamma}^{N_\Gamma+m}) \leq tol$).

It should be noted that every time a new backoff is computed the policy is re-optimized. This may look inefficient at first glance, however the convergence is achieved fast, as at every iteration, the previous policy is used as an initial guess for the next iteration, making the first iteration the most expensive one. Additionally, because the problem is treated as an expensive black-box optimization problem the number of iterations from this outer loop is in general quite small (in our examples no more than 10 iterations). (see Section 4.3). By the end of Algorithm 2, a probabilistically constrained policy will have been constructed.

Algorithm 2 Backoff-Based Policy Optimization

Input: Initialize policy parameter $\theta := \theta_0$, initial learning rate ℓ_0, learning rate update rule $L(\ell_k)$, define initial data matrix $\boldsymbol{\Gamma} := [\boldsymbol{\gamma}^1, \ldots, \boldsymbol{\gamma}^{N_\Gamma}]$ with N_Γ values for $\boldsymbol{\gamma}$, $0 < \delta < 1$, $0 < \alpha < 1$, tolerance tol_0, maximum number of backoff iterations M and S number of Monte Carlo samples to compute $\hat{F}_{lb}$, and number of epochs K.

1. Perform policy optimization with $\mathbf{b} = \mathbf{0}$ using Algorithm 1, to obtain nominal policy $\overline{\pi}_{\theta^*}$.

2. Estimate initial backoffs using S samples generated by Monte-Carlo simulations from the state trajectories of the nominal policy:
$b^0_{j,t} := F^{-1}_{\mathbf{g}_{j,t}}(1-\delta) - \bar{g}_{j,t}(\mathbf{x}_t),\ \forall j, t$

3. (i) Use the initially proposed set of backoff parameter values $\boldsymbol{\Gamma} = [\boldsymbol{\gamma}^1, \ldots, \boldsymbol{\gamma}^{N_\Gamma}]$ (*Notice* that the backoffs are $b_{j,t} := \gamma_j\, b^0_{j,t}\ \forall j, t$) to obtain optimal policies $\pi_{\theta^*}(\cdot|\boldsymbol{\gamma}^1) \ldots \pi_{\theta^*}(\cdot|\boldsymbol{\gamma}^{N_\Gamma})$

(ii) Evaluate the policies $\pi_{\theta^*}(\cdot|\boldsymbol{\gamma}^1) \ldots \pi_{\theta^*}(\cdot|\boldsymbol{\gamma}^{N_\Gamma})$ and compute their corresponding lower bounds $[\hat{F}_{lb}(\boldsymbol{\gamma}^1), \ldots, \hat{F}_{lb}(\boldsymbol{\gamma}^{N_\Gamma})]$ by Monte-Carlo using Eq. (19), and compute their residuals $\mathbf{F} = [\mathcal{F}(\boldsymbol{\gamma}^1), \ldots, \mathcal{F}(\boldsymbol{\gamma}^{N_\Gamma})]$ using Eq. (26). Notice that here, $\hat{F}_{lb}(\boldsymbol{\gamma})$ are only evaluated, without solving Eq. (19).

4. Perform Bayesian Optimization:

for $m = 1$ **to** ... **do**

i) Construct a mapping from the backoffs parameter values $\boldsymbol{\Gamma} = [\boldsymbol{\gamma}^1, \ldots, \boldsymbol{\gamma}^{N_\Gamma+m-1}]$ to their residuals $\mathbf{F} = [\mathcal{F}(\boldsymbol{\gamma}^1), \ldots, \mathcal{F}(\boldsymbol{\gamma}^{N_\Gamma+m-1})]$ by using a GP regression.

ii) Perform Bayesian optimization over the GP to minimize $\mathcal{F}(\boldsymbol{\gamma}^{N_\Gamma+m}) := \left(\hat{F}_{lb}(\boldsymbol{\gamma}^{N_\Gamma+m}) - (1-\alpha)\right)^2$.

iii) Perform policy optimization with $b_{j,t} := \gamma_j^{N_\Gamma+m}\, b^0_{j,t}\ \forall j, t$ using Algorithm 1, to obtain policy $\pi_{\theta^*}(\cdot|\boldsymbol{\gamma}^{N_\Gamma+m})$.

iv) Update data matrices $\boldsymbol{\Gamma} := [\boldsymbol{\Gamma}, \boldsymbol{\gamma}^{N_\Gamma+m}]$ and $\mathbf{F} := [\mathbf{F}, \mathcal{F}(\boldsymbol{\gamma}^{N_\Gamma+m})]$ by Monte-Carlo using Eq. (19). Notice that here, $\hat{F}_{lb}(\boldsymbol{\gamma})$ are only evaluated, without solving Eq. (19).

if $\mathcal{F}(\boldsymbol{\gamma}^{N_\Gamma+m}) \leq tol_0$ **then**

exit

end if

end for

Output: policy $\pi^*_\theta := \pi^{N_\Gamma+m}_{\theta^*}$.

4.3. Policy initialization

Reinforcement learning methods (particularly policy gradient) are computationally expensive; mainly because initially the agent (or controller in our case) explores the control action space randomly. In the case of process optimization and control, it is possible to use a preliminary controller, along with supervised learning to hot-start the policy, and significantly speed-up convergence. The initial parametrization for the policy (before **Step (1)**) is trained in a supervised learning fashion where the states are the inputs and the control actions are the outputs. This has been further discussed in [42–44].

5. Case studies

The case studies in this paper focuses on the photo-production of phycocyanin synthesized by cyanobacterium *Arthrospira platensis*. The two case studies are separated regarding the type of uncertainty: The case study 1 considers parametric uncertainty and model is considered to be known. The second case study considers that only data is available, where additive disturbance and measurement noise are present (no knowledge of the system's equations). Additionally, in the two case studies different penalizations of the constraints are implemented: Case study 1 uses Eq. (20) with arbitrary $\kappa = 0.1$ and $p = 1$, and Case study 2 uses Eq. (20) with arbitrary $\kappa = 1.$ and $p = 2$.

Phycocyanin is a high-value bioproduct and its biological function is to enhance the photosynthetic efficiency of cyanobacteria and red algae. It has applications as a natural colourant to replace other toxic synthetic pigments in both food and cosmetic production. Additionally, the pharmaceutical industry considers it beneficial because of its unique antioxidant, neuroprotective, and anti-inflammatory properties.

The dynamic system consists of the following system of ODEs describing the evolution of the concentration (c) of biomass (x), nitrate (N), and product (q). The dynamic model is based on Monod kinetics, which describes microorganism growth in nutrient sufficient cultures, where intracellular nutrient concentration is kept constant because of the rapid replenishment. We assume a fixed volume fed-batch. The manipulated variables as in the previous examples are the light intensity ($u_1 = I$) and inflow rate ($u_2 = F_N$). The mass balance equations are

$$\frac{dc_x}{dt} = u_m \frac{I}{I + k_s + I^2/k_i} \frac{c_x c_N}{c_N + K_N} - u_d c_X \tag{27}$$

$$\frac{dc_N}{dt} = -Y_{N/X} \frac{u_m I}{I + k_s + I^2/k_i} \frac{c_x c_N}{c_N + K_N} + F_N \tag{28}$$

$$\frac{dc_q}{dt} = \frac{k_m\, I}{I + k_{sq} + I^2/k_{iq}} c_x - \frac{k_d c_q}{C_N + K_{N_q}} \tag{29}$$

The parameter values are adopted from [22].

5.1. Case study 1

Uncertainty is assumed for the initial concentration, where $\begin{bmatrix} c_x(0) & c_N(0) \end{bmatrix} \sim \mathcal{N}(\begin{bmatrix} 1. & 150. \end{bmatrix}, \text{diag}(1\times10^{-3}, 22.5))$ and $c_q(0) = 0$. Additionally, 10% of parametric uncertainty for the system is assumed: $\frac{k_s}{(\mu\text{mol/m}^2\text{/s})} \sim \mathcal{N}(178.9, 17.89)$, $\frac{k_i}{(\text{mg/L})} \sim \mathcal{N}(447.1, 44.71)$, $\frac{k_N}{(\mu\text{mol/m}^2\text{/s})} \sim \mathcal{N}(393.1, 39.31)$. This type of uncertainty is common in engineering settings, as the parameters are obtained using experimental data, and they are subject to their respective confidence regions after they are estimated using regression techniques. The objective function (reward) in this work is to maximize the product's concentration (c_q) at the end of the batch. The objective is additionally penalized by the change of the control actions $\mathbf{u}(t) = [I, F_N]^T$. As a result the reward is:

$$\begin{aligned} &R_t = -\|\Delta \mathbf{u}_t\|_r,\ R_T = c_q(T) \\ &r = diag(3.125 \times 10^{-8}, 3.125 \times 10^{-6}) \end{aligned} \tag{30}$$

where $t \in \{0, T-1\}$ and $\Delta \mathbf{u}_t = \mathbf{u}_t - \mathbf{u}_{t-1}$. The constraints in this work for each time step are $c_N \leq 800$ and $c_q \leq 0.011 c_X$. These constraints have been normalized as:

$$g_{1,t} = \frac{c_N}{800} - 1 \leq 0, \quad g_{2,t} = \frac{c_q}{0.011 c_X} - 1 \leq 0 \tag{31}$$

and the joint chance constraint is meant to be satisfied with probability 99% ($\alpha = 0.01$) and confidence level is 99% ($\epsilon = 0.01$).

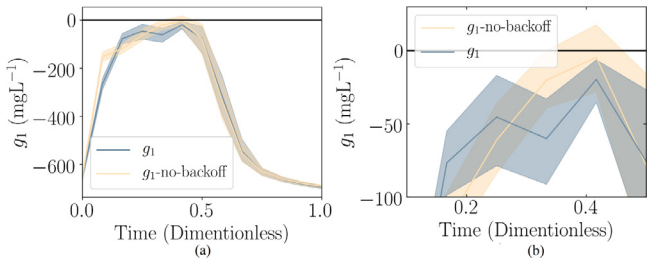

Fig. 1. Case study 1: Constraints when backoffs are applied (blue) and when there are set to be zero (yellow) for $g_{1,t}$ (a) and zoomed in the region that the violation of constraints occur (b). The shaded areas are the 98% and 2% percentiles.

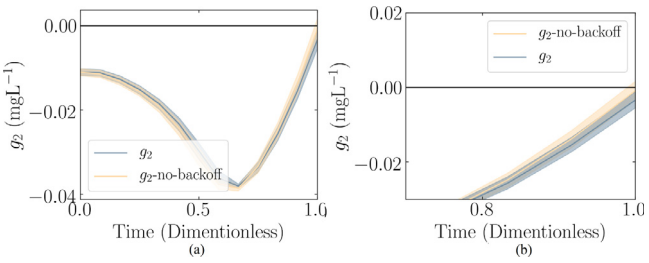

Fig. 2. Case study 1: Constraints when backoffs are applied (blue) and when there are set to be zero (yellow) for $g_{2,t}$ (a), zoomed region where the violation of constraints occurs (b). The shaded areas are the 98% and 2% percentiles.

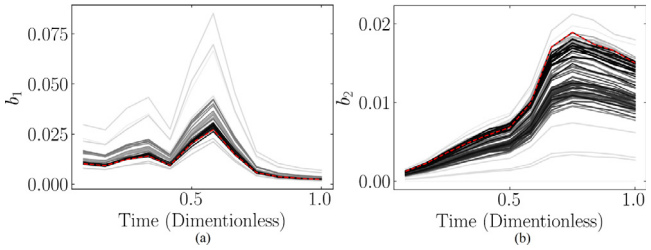

Fig. 3. Case study 2: The lines are plotted over number of epochs for $b_{1,t}$ (a) and, $b_{2,t}$ (b) which are faded out towards earlier iterations.

The constraints are added as a penalty with $\kappa = 1$ using (20) and $p = 1$. The control actions are constrained to be in the interval $0 \leq F_N \leq 40$ and $120 \leq I \leq 400$, these constraints are considered to be hard. The control policy RNN is designed to contain 4 hidden layers, each of which comprises 20 neurons with a leaky rectified linear unit (ReLU) as activation function. A unified policy network with diagonal variance is utilized such that the control actions share memory and the previous states are used from the RNN (together the current measured states). The computational cost for each control action online is insignificant since it only requires the evaluation of the corresponding RNN. First the algorithm computes the policy for the backoffs to be zero ($\mathbf{b} = 0$), then the backoffs are updated according to the Algorithm 2. The parameters for the trainings are: $M = 200$, $S = 1000$, $K = 200$, $tol = tol_0 = 10^{-4}$, $N_\gamma = 5$ and the two previous states and controls are used from the policy. Additionally, the Gaussian process has zero mean as prior, squared-exponential (SE) kernel as the covariance function, and the inputs-outputs are normalized using its mean and variance respectively. After the completion of the training the backoffs have been computed to satisfy (19). In no more than 10 iterations the backoff values managed to force the F_{lb} to 0.99.

Now, the actual closed-loop constraint satisfaction can be depicted in Fig. 1(a) and Fig. 2(a), where the shaded areas are the 98% and 2% percentiles. Notice, that even though the figures for both methods look similar the constraint satisfaction is significantly different, this can be observed in Fig. 1(b) and Fig. 2(b), where the region that the violation of constraints occurs has been zoomed in. This result is also quantitatively depicted in Table 1. The column labelled 'actual' is the probability of constraint violation that corresponds to the fraction of the 1000 Monte-Carlo trajectories that satisfied both of the constraints (see Eq. (14)). The column labelled 'desired' is the goal for the probability for constraint satisfaction. It is clear that when backoffs are not applied, almost 50% of the constraints are violated. On the other hand when backoffs are applied then all the constraints are satisfied, which is an expected result as the goal was the lower bound of ECDF (F_{lb}) to be 0.99, which means that the actual probability is equal or higher.

The backoff values for each update are shown in Fig. 3 (a, b), where the red-dashed represents the converged final value. It should be noted that the final value for the product c_q (30) is 0.163 and 0.167 when backoffs are applied and when they are not. This difference is to be expected given that in the nominal case ($\mathbf{b} = 0$), the policy allows the violation of constraints which lead to an increase in the concentration of the product.

5.2. Case study 2

The second case study considers the same system but this time no equation is assumed to be available. In this case, a Gaussian process [45] is used to model the available data. Different kernels can have different effects in the performance, in this case the Matérn32[45] kernel is employed. The data was generated using the system in Eqs. (27)–(29), and an additive normally distributed disturbance ($\mathbf{w}$) and measurement noise ($\mathbf{v}$) with zero mean and $\boldsymbol{\Sigma}_\mathbf{w} = \text{diag}(4\times10^{-4}, 0.1, 1\times10^{-8})$, $\boldsymbol{\Sigma}_\mathbf{v} = \text{diag}(4\times10^{-5}, 0.01, 1\times10^{-9})$. The same uncertainty for the initial conditions is assumed.

Bayesian frameworks have been used in reinforcement learning [46,47] as they can model both the epistemic and aleatoric uncertainty (in Gaussian processes, usually homoscedastic aleatoric uncertainty is considered). One of the fundamental steps here is the propagation of the uncertainty for each step. The nature of Gaussian processes has been exploited in [22,48,49], where the generated samples from the GP are conditioned in the posterior (without noise). Then, the GP predicts a mean $\boldsymbol{\mu}_\mathbf{x}$ and variance $\boldsymbol{\Sigma}_\mathbf{x}$ and the state $\mathbf{x}$ is sampled from a normal distribution: $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}_\mathbf{x}, \boldsymbol{\Sigma}_\mathbf{x})$. In this setting 8 episodes were randomly generated using Sobol sequences [50] for the control variables of all 12 time intervals and the initial conditions to train the Gaussian process.

After training, the actual closed-loop constraint satisfaction can be validated. Fig. 4(a) and Fig. 5(a) illustrate the path constraints for each time interval, where the shaded areas are the 98% and 2% percentiles. The results are compared with the case in absence of backoffs ($\mathbf{b} = 0$). It should be noticed that in the absence of backoffs the mean of the constraints' samples barely satisfies the bounds. The difference is clear and more apparent in Fig. 5(a). This can further be noticed in Fig. 4(b) and Fig. 5(b), where focus has been put on the region of constraint violation. Table 1 shows the probability of constraint violation that corresponds to the fraction of the 1000 Monte-Carlo trajectories that satisfied the constraints (see Eq. (14)) compared to the 'desired' probability for constraint satisfaction. The absence of constraint tightening results in only 24% of constraint satisfaction compare to our proposed method where 97% are satisfied. Notice that the desired probability designed in terms of the lower bound of the ECDF (F_{lb}) is set to be 0.95.

The backoff values for each update are shown in Fig. 6 (a, b), where the red-dashed represents the converged final value. It should be noted that the final value for the product c_q (30) is

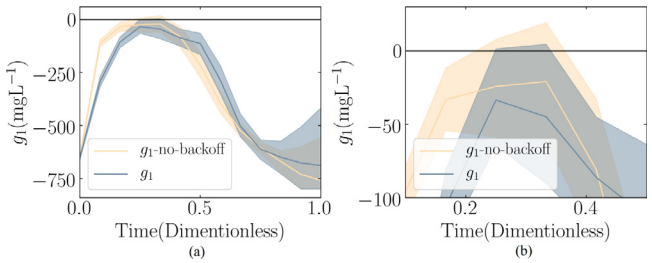

Fig. 4. Case study 2: Constraints when backoffs are applied (blue) and when they are set to be zero (yellow) for $g_{1,t}$ (a), zoomed region where the violation of constraints occurs (b). The shaded areas are the 98% and 2% percentiles.

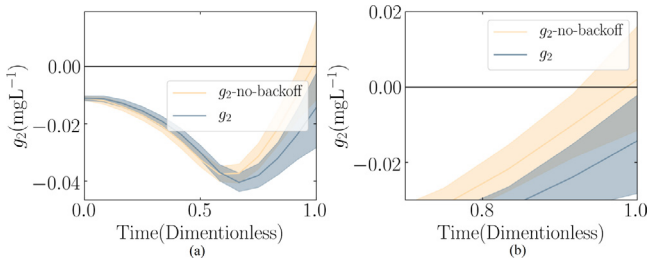

Fig. 5. Case study 2: Constraints when backoffs are applied (blue) and when they are set to zero (yellow) for $g_{2,t}$ (a), zoomed region where the violation of constraints occurs (b). The shaded areas are the 98% and 2% percentiles.

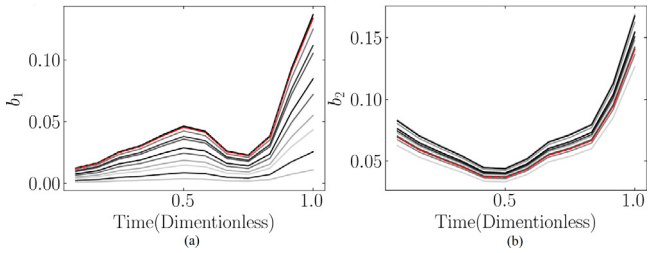

Fig. 6. Case study 2: The lines are plotted over number of epochs for $b_{1,t}$ (a) and, $b_{2,t}$ (b) which are faded out towards earlier iterations.

Table 1
Comparison of closed-loop constraint satisfaction. The symbol * corresponds to the cases that the constraint satisfaction is less than the desired.

Case Study	Desired	Actual
Case Study 1: Parametric Nominal	0.99	0.51*
Case Study 1: Parametric Proposed	0.99	1.00
Case Study 2: GP Nominal	0.95	0.24*
Case Study 2: GP Proposed	0.95	0.97

0.153 and 0.171 when backoffs are applied and when they are not.

The algorithm is implemented in Pytorch [51] version 0.4.1. Adam [52] is employed to compute the network's parameter values using a step size of 10^{-2} with the rest of hyperparameters at their default values.

6. Conclusions

In this paper we address the problem of finding a policy that can satisfy constraints with high probability. The proposed algorithm – chance constrained policy optimization (CCPO) – uses constraint tightening by applying backoffs to the original feasible set. Backoffs restrict the perceived feasible space by the controller, and allow guarantees on the satisfaction of chance constraints. We find the smallest backoffs (least conservative) that still guarantee the desired probability of satisfaction by stating the root-finding problem as a black-box optimization problem. This allows the algorithm to construct a policy that can guarantee the satisfaction of joint chance constraints with a user-defined probability of at least $1-\alpha$ and a user-defined confidence of at least $1-\epsilon$. Furthermore, the proposed methodology can be combined with other penalty or Lagrangian approaches for constrained policy search. Additionally, the first and second case study took 3hr and 4 h for offline computations, respectively. The online computational time of the policy is a few milliseconds.

Being able to solve constraint policy optimization problems with high probability satisfaction has been one of the main bottlenecks for the wider use of reinforcement learning in engineering applications. This work aims to take a step towards applying RL to the real world, where constraints on policies are necessary for the sake of safety and product quality.

Table 2
Nomenclature.

Symbol	Description
N_x	Size of variable $\mathbf{x}_t$
$\mathbf{x}_t$	State at time t
$\mathbf{u}_t$	Manipulated variables at time t
$\mathbf{w}_t$	External Disturbances at time t
$p(a\|b)$	Probability of event a given b
$\mathcal{N}(\cdot,\cdot)$	Normal distribution defined by mean and covariance
$\sim$	Distributed according to; example: $x \sim \mathcal{N}(\mu, \sigma^2)$
$\mathcal{D}$	Data available
$\mathbb{E}$	Expectation
$\mathbb{P}$	Probability
$\mathbb{X}$	Feasible space for states variables
$\mathbb{U}$	Feasible space for manipulated variables
$\cap$	Intersection of sets
γ	Discount factor
R_t	Reward at time t
g_j, t	Constraint j at time t
$\pi_\theta(\cdot)$	Policy parametrized by θ
b_j, t	Backoff of constraint j at time t
$\boldsymbol{\tau}$	Joint random variable of states, controls and reward
$1-\alpha$	The probability of constraint satisfaction
$1-epsilon$	Confidence level
$F(c)$	Cumulative distribution function (CDF) at c
$\mathbb{1}(\cdot)$	Indicator function
Bin	Binomial distribution
F_S	Cumulative distribution function (ECDF) for F with S samples
$\hat{F}_S$	Realization of F_S
betainv	Inverse cumulative distribution of Beta probability distribution
F_{lb}	Lower bound of ECDF F_S
\hatF_lb	Realization of F_lb
pmb\gamma	Parametrization of backoffs
$\|\|\cdot\|\|_p$	l_p norm
∇_θ	Partial derivatives with respect to θ

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This project has received funding from the EPSRC projects, UK (EP/R032807/1) and (EP/P016650/1).

Appendix A. Bayesian optimization

Bayesian optimization is a popular approach for black-box optimization [26]. A review of Bayesian optimization can be found in [53].

In this paper Bayesian optimization is exploited to obtain the minimizer $\boldsymbol{\gamma}^*$ of $\mathcal{F}(\boldsymbol{\gamma}) = \left(\hat{F}_{lb} - (1-\alpha)\right)^2$:

$$\boldsymbol{\gamma}^* \in \arg\min_{\boldsymbol{\gamma}} \mathcal{F}(\boldsymbol{\gamma}) \tag{32}$$

The function $\mathcal{F}(\boldsymbol{\gamma})$ to be minimized can only be observed through an unbiased noisy observation based on MC estimates of $\hat{F}_{lb}$:

$$y = \mathcal{F}(\mathbf{x}) + \omega \tag{33}$$

where ω is assumed to be Gaussian white noise. The noise ω is assumed to be unknown and estimated within the GP framework. Commonly Gaussian processes (GP) are employed as nonparametric models. To start the algorithm we first require some data to build the initial GP, i.e. the function $\mathcal{F}(\cdot)$ is queried at N_Γ points. In general the input data-points $\boldsymbol{\Gamma} = [\boldsymbol{\gamma}_1, \ldots, \boldsymbol{\gamma}_{N_\Gamma}]$ are selected based on a space-filling design, such as a Latin hypercube design [54]. From this we obtain the corresponding responses $\hat{\mathcal{F}}_{lb} = [\mathcal{F}(\boldsymbol{\gamma}^1), \ldots, \mathcal{F}(\boldsymbol{\gamma}^{N_\Gamma})]$. A GP model can then be trained from the input–output data. In particular the hyperparameters of the GP were trained using maximum likelihood estimation. The GP model can then be utilized to obtain the Gaussian distribution of $\mathcal{F}(\boldsymbol{\gamma})$ at an arbitrary query point $\boldsymbol{\gamma}$ [45]:

$$\mathcal{F}(\boldsymbol{\gamma})|\boldsymbol{\Gamma}, \hat{\mathcal{F}}_{lb} \sim \mathcal{N}\left(\mu_{GP}(\boldsymbol{\gamma}), \sigma^2_{GP}(\boldsymbol{\gamma})\right) \tag{34}$$

where $\mu_{GP}(\boldsymbol{\gamma})$ and $\sigma^2_{GP}(\boldsymbol{\gamma})$ are the mean and variance prediction of the GP respectively. In this setting the approach sequentially selects a location $\boldsymbol{\gamma}$ at which to query $\mathcal{F}(\cdot)$ and observe y. After a selected number of iterations M the algorithm returns a best-estimate of $\boldsymbol{\gamma}^*$. To accomplish this the GP of $\mathcal{F}(\cdot)$ is iteratively updated from the available data of $\mathcal{F}(\cdot)$. One could simply sample at the minimum of the mean function $\mu_{GP}(\boldsymbol{\gamma})$. However, sampling at a point with higher uncertainty could yield a lower minimum, i.e. there is a trade-off between sampling at points with low values of the mean function $\mu_{GP}(\boldsymbol{\gamma})$ and high values of the variance function $\sigma^2_{GP}(\boldsymbol{\gamma})$. The selection of the query points is given by so-called acquisition function, for which we used the lower confidence bound:

$$\boldsymbol{\gamma}_m = \arg\min_{\boldsymbol{\gamma}} \mu_{GP}(\boldsymbol{\gamma}) - 3\sigma_{GP}(\boldsymbol{\gamma}) \tag{35}$$

where $\boldsymbol{\gamma}_m$ denotes the query point at iteration m. Note that the query point at each iteration are chosen at points that are predicted to be low by the mean function, but could also potentially yield lower values according to the variance function.

Appendix B. Control actions

In this section the control actions are presented for completeness.

B.1. Case study 1

See Figs. 7 and 8.

B.2. Case study 2

See Figs. 9 and 10.

Appendix C. Nomenclature

See Table 2.

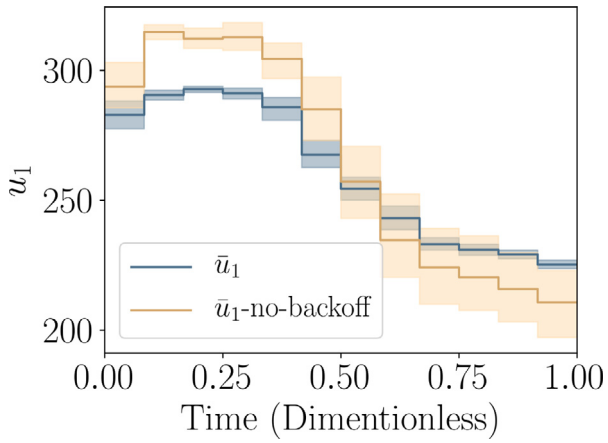

Fig. 7. Case study 1: Control actions u_1 when backoffs are applied (blue) and when there are set to be zero (yellow). The shaded areas are the 98% and 2% percentiles.

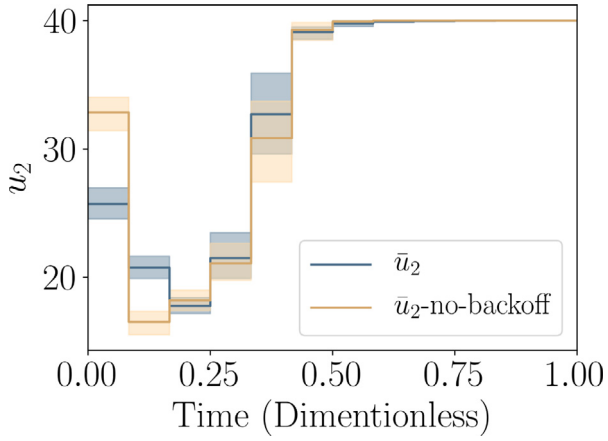

Fig. 8. Case study 1: Control actions u_1 when backoffs are applied (blue) and when there are set to be zero (yellow). The shaded areas are the 98% and 2% percentiles.

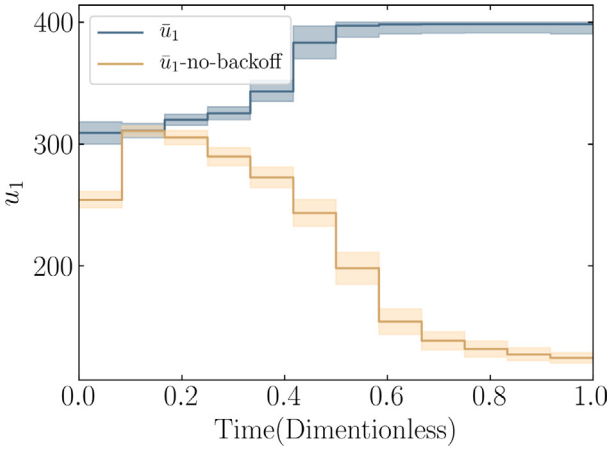

Fig. 9. Case study 2: Control actions u_1 when backoffs are applied (blue) and when there are set to be zero (yellow). The shaded areas are the 98% and 2% percentiles.

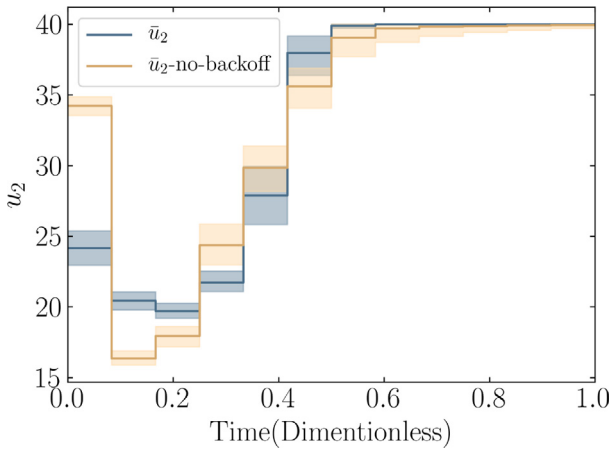

Fig. 10. Case study 2: Control actions u_1 when backoffs are applied (blue) and when there are set to be zero (yellow). The shaded areas are the 98% and 2% percentiles.

References

[1] D.P. Bertsekas, Dynamic Programming and Optimal Control, second ed., Athena Scientific, 2000.

[2] S. Spielberg, A. Tulsyan, N.P. Lawrence, P.D. Loewen, R. Bhushan Gopaluni, Toward self-driving processes: A deep reinforcement learning approach to control, AIChE J. 65 (10) (2019) e16689.

[3] J.M. Lee, J.H. Lee, Approximate dynamic programming-based approaches for input–output data-driven control of nonlinear processes, Automatica 41 (7) (2005) 1281–1288.

[4] C. Peroni, N. Kaisare, J. Lee, Optimal control of a fed-batch bioreactor using simulation-based approximate dynamic programming, IEEE Trans. Control Syst. Technol. 13 (5) (2005) 786–790.

[5] J.H. Lee, J.M. Lee, Approximate dynamic programming based approach to process control and scheduling, Comput. Chem. Eng. 30 (10–12) (2006) 1603–1618.

[6] W. Tang, P. Daoutidis, Distributed adaptive dynamic programming for data-driven optimal control, Systems Control Lett. 120 (2018) 36–43.

[7] D. Chaffart, L.A. Ricardez-Sandoval, Optimization and control of a thin film growth process: A hybrid first principles/artificial neural network based multiscale modelling approach, Comput. Chem. Eng. 119 (2018) 465–479.

[8] H. Shah, M. Gopal, Model-free predictive control of nonlinear processes based on reinforcement learning, IFAC-PapersOnLine 49 (1) (2016) 89–94.

[9] A. Bemporad, M. Morari, Robust model predictive control: A survey, Robust. Identif. Control 245 (1999) 207–226.

[10] V.M. Charitopoulos, V. Dua, Explicit model predictive control of hybrid systems and multiparametric mixed integer polynomial programming, AIChE J. 62 (9) (2016) 3441–3460, [Online]. Available: https://aiche.onlinelibrary.wiley.com/doi/abs/10.1002/aic.15396.

[11] R.S. Sutton, D. McAllester, S. Singh, Y. Mansour, Policy gradient methods for reinforcement learning with function approximation, in: Proceedings Of The 12th International Conference On Neural Information Processing Systems, in: NIPS'99, MIT Press, Cambridge, MA, USA, 1999, pp. 1057–1063.

[12] M. Wen, Constrained cross-entropy method for safe reinforcement learning, Neural Inf. Process. Syst. (NIPS) (Nips) (2018).

[13] J. Achiam, D. Held, A. Tamar, P. Abbeel, Constrained policy optimization, 2017, arxiv preprint arXiv:1705.10528.

[14] C. Tessler, D.J. Mankowitz, S. Mannor, Reward constrained policy optimization, no. 2016, pp. 1–15, 2018, arxiv preprint arxiv:1805.11074.

[15] Y. Chow, O. Nachum, A. Faust, E. Duenez-Guzman, M. Ghavamzadeh, Lyapunov-Based safe policy optimization for continuous control, 2019, [Online]. Available: http://arxiv.org/abs/1901.10031.

[16] T.-Y. Yang, J. Rosca, K. Narasimhan, P.J. Ramadge, Projection-based constrained policy optimization, in: International Conference On Learning Representations, 2020, [Online]. Available: https://openreview.net/forum?id=rke3TJrtPS.

[17] Y. Liu, J. Ding, X. Liu, IPO: Interior-point policy optimization under constraints, 2019, [Online]. Available: http://arxiv.org/abs/1910.09615.

[18] P. Petsagkourakis, W.P. Heath, J. Carrasco, C. Theodoropoulos, Robust stability of barrier-based model predictive control, IEEE Trans. Autom. Control (2020) http://dx.doi.org/10.1109/TAC.2020.3010770.

[19] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, A. Madry, Implementation matters in deep RL: A case study on PPO and TRPO, in: International Conference On Learning Representations, 2020, [Online]. Available: https://openreview.net/forum?id=r1etN1rtPB.

[20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, arXiv preprint arXiv:1707.06347.

[21] P. Petsagkourakis, I. Sandoval, E. Bradford, D. Zhang, E. del Rio-Chanona, Reinforcement learning for batch bioprocess optimization, Comput. Chem. Eng. 133 (2020) 106649.

[22] E. Bradford, L. Imsland, D. Zhang, E.A. del Rio Chanona, Stochastic data-driven model predictive control using gaussian processes, Comput. Chem. Eng. 139 (2020) 106844.

[23] M. Rafiei, L.A. Ricardez-Sandoval, Stochastic back-off approach for integration of design and control under uncertainty, Ind. Eng. Chem. Res. 57 (12) (2018) 4351–4365, http://dx.doi.org/10.1021/acs.iecr.7b03935.

[24] M.P. Deisenroth, D. Fox, C.E. Rasmussen, Gaussian Processes for data-efficient learning in robotics and control, IEEE Trans. Pattern Anal. Mach. Intell. 37 (2) (2015).

[25] D. Zhan, H. Xing, Expected improvement for expensive optimization: a review, J. Global Optim. (2020) http://dx.doi.org/10.1007/s10898-020-00923-x.

[26] D.R. Jones, M. Schonlau, W.J. Welch, Efficient global optimization of expensive black-box functions, J. Global Optim. 13 (4) (1998) 455–492.

[27] C. Cartis, L. Roberts, O. Sheridan-Methven, Escaping local minima with derivative-free methods: a numerical investigation, 2018.

[28] D.E. Rumelhart, G.E. Hinton, R.J. Williams, Learning representations by back-propagating errors, Nature 323 (1986) 533.

[29] K. Greff, R.K. Srivastava, J. Koutník, B.R. Steunebrink, J. Schmidhuber, LSTM: A search space odyssey, IEEE Trans. Neural Netw. Learn. Syst. 28 (10) (2017) 2222–2232.

[30] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing atari with deep reinforcement learning, 2013, pp. 1–9, arxiv preprint arxiv:1312.5602.

[31] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, D. Hassabis, Human-level control through deep reinforcement learning, Nature 518 (2015) 529.

[32] C.J. Clopper, E.S. Pearson, The use of confidence or fiducial limits illustrated in the case of the binomial, Biometrika 26 (4) (1934) 404–413.

[33] L.D. Brown, T.T. Cai, A. DasGupta, Interval estimation for a binomial proportion, Statist. Sci. (2001) 101–117.

[34] J.A. Paulson, A. Mesbah, Nonlinear model predictive control with explicit backoffs for stochastic systems under arbitrary uncertainty, IFAC-PapersOnLine 51 (20) (2018) 523–534.

[35] R.J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, Mach. Learn. 8 (3–4) (1992) 229–256.

[36] S. Kakade, A natural policy gradient, Adv. Neural Inf. Process. Syst. (2002).

[37] J. Nocedal, S.J. Wright, Numerical Optimization, second ed., Springer, New York, NY, USA, 2006.

[38] S.J. Wright, Primal-Dual Interior-Point Methods, Society for Industrial and Applied Mathematics, 1997, [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9781611971453.

[39] M.C. Campi, S. Garatti, F.A. Ramponi, A general scenario theory for non-convex optimization and decision making, IEEE Trans. Autom. Control 63 (12) (2018) 4067–4078.

[40] R. Sutton, A. Barto, Reinforcement Learning: An Introduction, second ed., MIT Press, 2018.

[41] P.I. Frazier, A tutorial on Bayesian optimization, 2018, arXiv preprint arXiv:1807.02811.

[42] P. Petsagkourakis, I.O. Sandoval, E. Bradford, D. Zhang, E.A. del Río Chanona, Constrained reinforcement learning for dynamic optimization under uncertainty, 2020.

[43] J.A. Paulson, A. Mesbah, Approximate closed-loop robust model predictive control with guaranteed stability and constraint satisfaction, IEEE Control Syst. Lett. 4 (3) (2020) 719–724.

[44] B. Karg, S. Lucia, Efficient representation and approximation of model predictive control laws via deep learning, IEEE Trans. Cybern. (2020) 1–13.

[45] C. Rasmussen, C. Williams, Gaussian Processes For Machine Learning, in: Adaptive Computation and Machine Learning, MIT Press, Max-Planck-Gesellschaft, Biologische Kybernetik, Cambridge, MA, USA, 2006, p. 248.

[46] K. Chua, R. Calandra, R. Mcallister, S. Levine, Deep reinforcement learning in a handful of trials using probabilistic dynamics models.

[47] M. Janner, J. Fu, M. Zhang, S. Levine, When to trust your model: Model-based policy optimization.

[48] J. Umlauft, T. Beckers, S. Hirche, Scenario-based optimal control for Gaussian process state space models, in: 2018 European Control Conference, ECC 2018, European Control Association (EUCA), 2018, pp. 1386–1392.

[49] L. Hewing, E. Arcari, L.P. Fröhlich, M.N. Zeilinger, On simulation and trajectory prediction with Gaussian process dynamics, 2019, [Online]. Available: http://arxiv.org/abs/1912.10900.

[50] I. Sobol, Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates, Math. Comput. Simul. 55 (1) (2001) 271–280, The Second IMACS Seminar on Monte Carlo Methods.

[51] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, Pytorch: An imperative style, high-performance deep learning library, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d. Alché-Buc, E. Fox, R. Garnett (Eds.), Advances In Neural Information Processing Systems, Vol. 32, 2019, pp. 8024–8035.

[52] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014.

[53] B. Shahriari, K. Swersky, Z. Wang, R.P. Adams, N. de Freitas, Taking the human out of the loop: A review of Bayesian optimization, Proc. IEEE 104 (1) (2016) 148–175.

[54] R.L. Iman, J.C. Helton, J.E. Campbell, An approach to sensitivity analysis of computer models: Part I—Introduction, input variable selection and preliminary variable assessment, J. Qual. Technol. 13 (3) (1981) 174–183.